

Troubleshoot Common ODE Problems

This topic describes common issues you might encounter when solving ODEs with solver functions (ode45, ode15s, and so on).

Error Tolerances

Question or Problem	Answer
How do I choose the error thresholds RelTol and AbsTol?	RelTol, the relative accuracy tolerance, controls the number of correct digits in the computed answer. AbsTol, the absolute error tolerance, controls the difference between the computed answer and the true solution. At each step, the error e in component i of the solution satisfies $ e(i) \leq \max(\text{RelTol} * \text{abs}(y(i)), \text{AbsTol}(i))$ Roughly speaking, this means that you want RelTol correct digits in all solution components, excluding those smaller than the threshold AbsTol(i). Even if you are not interested in a component $y(i)$ when it is small, you might have to specify a value for AbsTol(i) that is small enough to get some correct digits in $y(i)$ so that you can accurately compute more interesting components.
I want answers that are correct to the precision of the computer. Why can I not simply set RelTol to eps?	You can get close to machine precision, but not that close. The solvers do not allow RelTol near eps because they try to approximate a continuous function. At tolerances comparable to eps, the machine arithmetic causes all functions to look discontinuous.
How do I tell the solver that I do not care about getting an accurate answer for one of the solution components?	You can increase the absolute error tolerance AbsTol for this solution component. If the tolerance is bigger than the solution component, this specifies that no digits in the component need to be correct. The solver might have to get some correct digits in this component to compute other components accurately, but it generally handles this automatically.

Problem Scale

Question or Problem	Answer
How large a problem can I solve with the ODE suite?	The primary constraints are memory and time. At each time step, the solvers for nonstiff problems allocate vectors of length n , where n is the number of equations in the system. The solvers for stiff problems allocate vectors of length n but also allocate an n -by- n Jacobian matrix. For these solvers,

Question or Problem	Answer
	it might be advantageous to specify the Jacobian sparsity pattern using the JPattern option of odeset. If the problem is nonstiff, or if you are using the JPattern option, it might be possible to solve a problem with thousands of unknowns. In this case, however, storage of the result can be problematic. Ask the solver to evaluate the solution at specific points only, or call the solver with no output arguments and use an output function to monitor the solution.
I am solving a very large system, but only care about a few of the components of y . Is there any way to avoid storing all of the elements?	Yes. The OutputFcn option is designed specifically for this purpose. When you call the solver with no output arguments, the solver does not allocate storage to hold the entire solution history. Instead, the solver calls <code>OutputFcn(t,y,flag)</code> at each time step. To keep the history of specific elements, write an output function that stores or plots only the elements you care about.
What is the startup cost of the integration and how can I reduce it?	The biggest startup cost occurs as the solver attempts to find a step size appropriate to the scale of the problem. If you happen to know an appropriate step size, use the InitialStep option. For example, if you repeatedly call the integrator in an event location loop, the last step that was taken before the event is probably scaled correctly for the next integration. Type <code>edit ballode</code> to see an example.
The first step size that the integrator takes is too large, and it misses important behavior.	You can specify the first step size with the InitialStep option. The integrator tries this step size, then reduces it if necessary.

Solution Components

Question or Problem	Answer
The solution does not look like what I expected.	If your expectations are correct, then reduce the error tolerances from their default values. A smaller relative error tolerance is needed to accurately solve problems integrated over “long” intervals, as well as problems that are moderately unstable. Check whether there are solution components that stay smaller than their absolute error tolerance for some time. If so, you are not requiring any correct digits in these components. This might be acceptable for these components, but failing to compute them accurately might degrade the accuracy of other components that depend on them.

Question or Problem	Answer
My plots are not smooth enough.	Increase the value of Refine from its default value of 4 in ode45 , 8 in ode78 and ode89 , and 1 in the other solvers. The larger the value of Refine, the more output points the solver generates. Execution speed is not affected much by increasing the value of Refine from its default value.
I am plotting the solution as it is computed and it looks fine, but the code gets stuck at some point.	First verify that the ODE function is smooth near the point where the code gets stuck. If it is not, then the solver must take small steps to deal with this. It might help to break the integration interval into pieces over which the ODE function is smooth. If the function is smooth and the code is taking extremely small steps, you are probably trying to solve a stiff problem with a solver not intended for that purpose. Switch to using one of the stiff solvers ode15s, ode23s, ode23t, or ode23tb.
What if I have the final and not the initial value?	All the solvers of the ODE suite allow you to solve backward or forward in time. The syntax for the solvers is <code>[t,y] = ode45(odefun,[t0 tf],y0);</code> and the syntax accepts <code>t0 > tf</code> .
My integration proceeds very slowly, using too many time steps.	First, check that <code>tspan</code> is not too long. Remember that the solver uses as many time points as necessary to produce a smooth solution. If the ODE function changes on a time scale that is very short compared to <code>tspan</code>, then the solver uses a lot of time steps. Long-time integration is a hard problem. Break <code>tspan</code> into smaller pieces. If the ODE function does not change noticeably on the <code>tspan</code> interval, it could be that your problem is stiff. Try using one of the stiff solvers ode15s, ode23s, ode23t, or ode23tb. Finally, make sure that the ODE function is written in an efficient way. The solvers evaluate the derivatives in the ODE function many times. The cost of numerical integration depends critically on the expense of evaluating the ODE function. Rather than recomputing complicated constant parameters at each evaluation, store them in global variables, or calculate them once and pass them to nested functions.

Question or Problem	Answer
I know that the solution undergoes a radical change at time t , where $t_0 \leq t \leq t_f$, but the integrator steps past without “seeing” it.	If you know that there is a sharp change at time t, try breaking the t_{span} interval into two pieces, $[t_0 \ t]$ and $[t \ t_f]$, and call the integrator twice or continue the integration using <code>odextend</code>. If the differential equation has periodic coefficients or solutions, ensure the solver does not step over periods by restricting the maximum step size to the length of the period.

Problem Type

Can the solvers handle partial differential equations (PDEs) that have been discretized by the method of lines?	Yes, because the discretization produces a system of ODEs. Depending on the discretization, you might have a form involving mass matrices, which the ODE solvers provide for. Often the system is stiff. This is to be expected if the PDE is parabolic, or when there are phenomena that happen on very different time scales such as a chemical reaction in a fluid flow. In such cases, use one of the four stiff solvers <code>ode15s</code>, <code>ode23s</code>, <code>ode23t</code>, or <code>ode23tb</code>. If there are many equations, use the <code>JPattern</code> option to specify the Jacobian sparsity pattern. This can make the difference between success and failure as it prevents the computation from being too expensive. Type <code>edit burgersode</code> to see an example that uses <code>JPattern</code>. If the system is not stiff, or not very stiff, then <code>ode23</code> and <code>ode45</code> are more efficient than the stiff solvers <code>ode15s</code>, <code>ode23s</code>, <code>ode23t</code>, and <code>ode23tb</code>. You can solve parabolic-elliptic partial differential equations in 1-D directly with the MATLAB® PDE solver <code>pdepe</code>.
Can I integrate a set of sampled data?	Not directly. Instead, represent the data as a function by interpolation or some other scheme for fitting data. The smoothness of this function is critical. A piecewise polynomial fit such as a spline can look smooth to the eye, but rough to a solver; the solver takes small steps where the derivatives of the fit have jumps. Either use a smooth function to represent the data or use one of the lower-order solvers (<code>ode23</code>, <code>ode23s</code>, <code>ode23t</code>, <code>ode23tb</code>) that is less sensitive to smoothness. See ODE with Time-Dependent Terms for an example.

See Also

[odeset](#) | [odeget](#) | [deval](#) | [odextend](#)

Topics

[Choose an ODE Solver](#)

[Summary of ODE Options](#)

[ODE Event Location](#)

[Nonnegative ODE Solution](#)